

Path 1: Pre Security > Module 3: How The Web Works > Room 3: How Websites Work

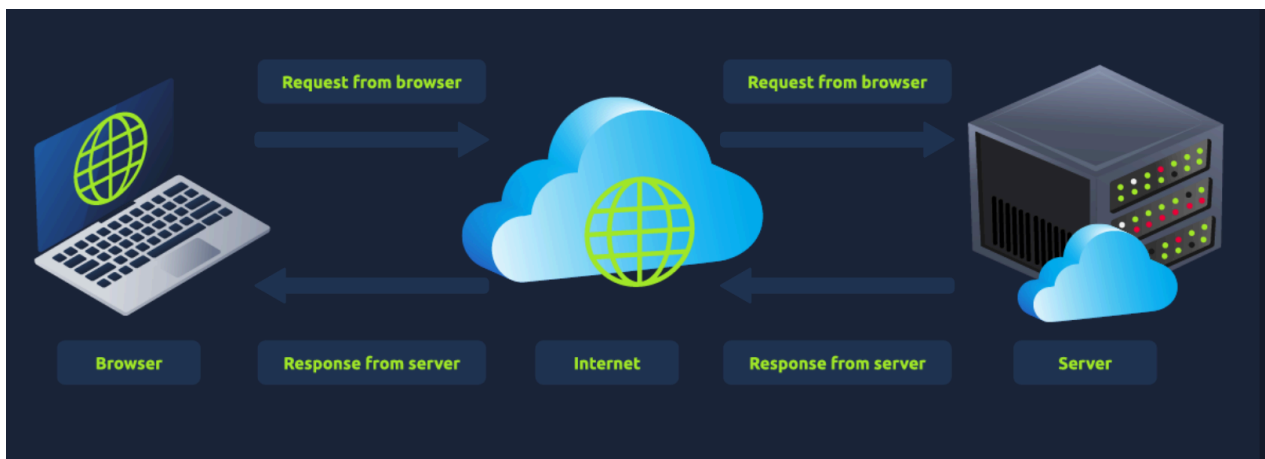
<https://tryhackme.com/room/howwebsiteswork>

How Websites Work:

To exploit a website, you first need to know how they are created.

>> Task 1: How websites work

When you visit a website, your browser (like Safari or Google Chrome) makes a request to a web server asking for information about the page you're visiting. It will respond with data that your browser uses to show you the page; a web server is just a dedicated computer somewhere else in the world that handles your requests.



There are two major components that make up a website:

1. Front End (Client-Side) - the way your browser renders a website.
2. Back End (Server-Side) - a server that processes your request and returns a response.

There are many other processes involved in your browser making a request to a web server, but for now, you just need to understand that you make a request to a server, and it responds with data your browser uses to render information to you.

Q1) What term best describes the component of a web application rendered by your browser?

Answer: Front End (Client Side)

>> Task 2: HTML

Websites are primarily created using:

- **HTML**, to build websites and define their structure
- **CSS**, to make websites look pretty by adding styling options
- **JavaScript**, implement complex features on pages using interactivity

HyperText Markup Language (HTML) is the language websites are written in. Elements (also known as tags) are the building blocks of HTML pages and tells the browser how to display content. The code snippet below shows a simple HTML document, the structure of which is the same for every website:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>Example Heading</h1>
    <p>Example paragraph..</p>
  </body>
</html>
```

The HTML structure (as shown in the screenshot) has the following components:

- The **<!DOCTYPE html>** defines that the page is a HTML5 document. This helps with standardisation across different browsers and tells the browser to use HTML5 to interpret the page.
- The **<html>** element is the root element of the HTML page - all other elements come after this element.
- The **<head>** element contains information about the page (such as the page title)
- The **<body>** element defines the HTML document's body; only content inside of the body is shown in the browser.
- The **<h1>** element defines a large heading
- The **<p>** element defines a paragraph
- There are many other elements (tags) used for different purposes. For example, there are tags for buttons (**<button>**), images (****), lists, and much more.

Tags can contain attributes such as the class attribute which can be used to style an element (e.g. make the tag a different color) `<p class="bold-text">`, or the src attribute which is used on images to specify the location of an image: ``. An element can have multiple attributes each with its own unique purpose, e.g., `<p attribute1="value1" attribute2="value2">`.

Elements can also have an id attribute (`<p id="example">`), which is unique to the element. Unlike the class attribute, where multiple elements can use the same class, an element must have different id's to identify them uniquely. Element id's are used for styling and to identify it by JavaScript.

You can view the HTML of any website by right-clicking and selecting "View Page Source" (Chrome) / "Show Page Source" (Safari).

Answer The Questions Below:

Let's play with some HTML! First click the "View Site" button inside this task. On the right-hand side, you should see a box that renders HTML - If you enter some HTML into the box and click the green "Render HTML Code" button, it will render your HTML on the page; you should see an image of some cats.

```
<!DOCTYPE html>
<html>
  <head>
    <title>TryHackMe HTML Editor</title>
  </head>
  <body>
    <h1>Cat Website!</h1>
    <p>See images of all my cats!</p>
    <img src='img/cat-1.jpg'>
    <img src='img/cat-2.'>
    <!-- Add dog image here -->
  </body>
</html>
```

Q1) One of the images on the cat website is broken - fix it, and the image will reveal the hidden text answer!

Answer: HTMLHERO

Q2) Add a dog image to the page by adding another img tag (`<img>`) on line 11. The dog image location is `img/dog-1.png`. What is the text in the dog image?

Answer: DOGHTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>TryHackMe HTML Editor</title>
  </head>
  <body>
    <h1>Cat Website!</h1>
    <p>See images of all my cats!</p>
    <img src='img/cat-1.jpg'>
    <img src='img/cat-2.jpg'>
    <img src='img/dog-1.png'>
  </body>
</html>
```

>> Task 3: JavaScript

JavaScript (JS) is one of the most popular coding languages in the world and allows pages to become interactive. HTML is used to create the website structure and content, while JavaScript is used to control the functionality of web pages - without JavaScript, a page would not have interactive elements and would always be static. JS can dynamically update the page in real-time, giving functionality to change the style of a button when a particular event on the page occurs (such as when a user clicks a button) or to display moving animations.

JavaScript is added within the page source code and can be either loaded within `<script>` tags or can be included remotely with the src attribute: `<script src="/location/of/javascript_file.js"></script>`

The following JavaScript code finds a HTML element on the page with the id of "demo" and changes the element's contents to "Hack the Planet" :

`document.getElementById("demo").innerHTML = "Hack the Planet";`

HTML elements can also have events, such as "onclick" or "onhover" that execute JavaScript when the event occurs. The following code changes the text of the element with the demo ID to Button Clicked:

`<button onclick='document.getElementById("demo").innerHTML = "Button Clicked";'>Click Me!</button>` - onclick events can also be defined inside the JavaScript script tags, and not on elements directly.

Answer the questions below:

```
<!DOCTYPE html>
<html>
  <head>
    <title>TryHackMe Editor</title>
  </head>
  <body>
    <div id="demo">Hi there!</div>
    <script type="text/javascript">
      // add your JavaScript here
    </script>
  </body>
</html>
```

Q1) lick the "View Site" button on this task. On the right-hand side, add JavaScript that changes the demo element's content to "Hack the Planet"

Answer: JSISFUN

```
<!DOCTYPE html>
<html>
  <head>
    <title>TryHackMe Editor</title>
  </head>
  <body>
    <div id="demo">Hi there!</div>
    <script type="text/javascript">
      document.getElementById("demo").innerHTML = "Hack the Planet";
    </script>
  </body>
</html>
```

Q2) Add the button HTML from this task that changes the element's text to "Button Clicked" on the editor on the right, update the code by clicking the "Render HTML+JS Code" button and then click the button.

Answer: No answer needed

```
<!DOCTYPE html>
<html>
  <head>
    <title>TryHackMe Editor</title>
  </head>
  <body>
    <div id="demo">Hi there!</div>
    <button onclick='document.getElementById("demo").innerHTML = "Hack the Planet";'>Click Me!</button>
    <script type="text/javascript">
      // add your JavaScript here
    </script>
  </body>
</html>
```

>> Task 4: Sensitive Data Exposure

Sensitive Data Exposure occurs when a website doesn't properly protect (or remove) sensitive clear-text information to the end-user; usually found in a site's frontend source code.

We now know that websites are built using many HTML elements (tags), all of which we can see simply by "viewing the page source". A website developer may have forgotten to remove login credentials, hidden links to private parts of the website or other sensitive data shown in HTML or JavaScript.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Fake Website</title>
  </head>
  <body>
    <form>
      <input type='text' name='username'>
      <input type='password' name='password'>
      <button>Login</button>
      <!-- TODO: remove test credentials admin:password123 -->
    </form>
  </body>
</html>
```

Sensitive information can be potentially leveraged to further an attacker's access within different parts of a web application. For example, there could be HTML comments with temporary login credentials, and if you viewed the page's source code and found this, you could use these credentials to log in elsewhere on the application (or worse, used to access other backend components of the site).

Whenever you're assessing a web application for security issues, one of the first things you should do is review the page source code to see if you can find any exposed login credentials or hidden links.

Q1) View the website on this link

(https://static-labs.tryhackme.cloud/sites/howwebsiteswork/html_data_exposure/).

What is the password hidden in the source code?

Answer: testpasswd

ctrl + U = To look Source Code

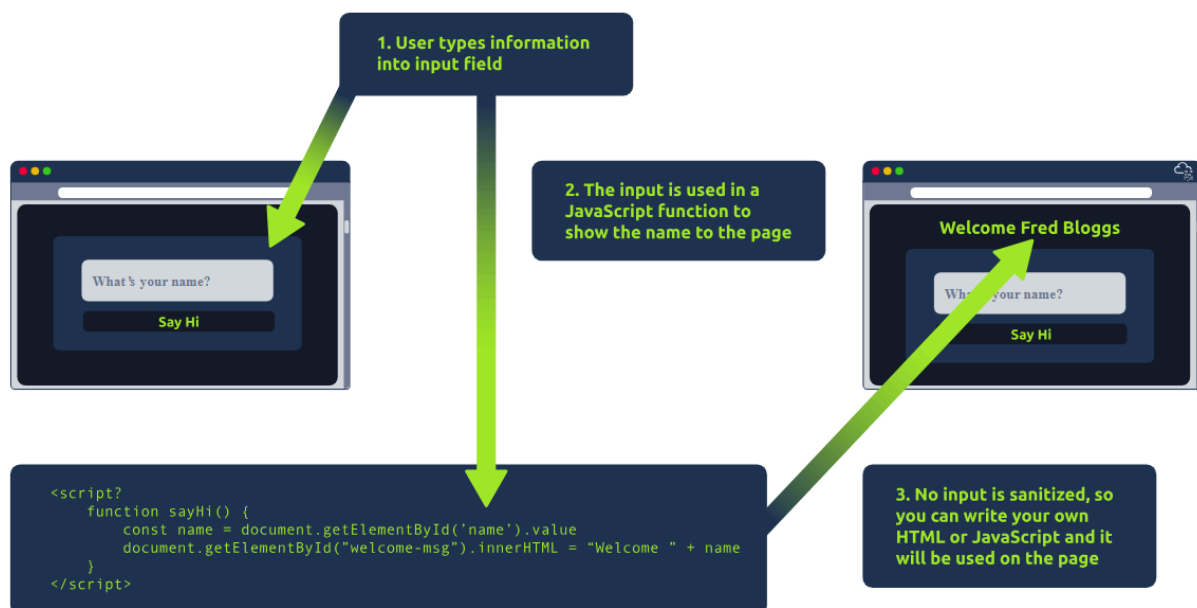
The link requires internet access, so you can not use it with a free AttackBox but instead open it from your own machine.

>> Task 5: HTML Injection

HTML Injection is a vulnerability that occurs when unfiltered user input is displayed on the page. If a website fails to sanitise user input (filter any "malicious" text that a user inputs into a website), and that input is used on the page, an attacker can inject HTML code into a vulnerable website.

Input sanitisation is very important in keeping a website secure, as information a user inputs into a website is often used in other frontend and backend functionality. A vulnerability you'll explore in another lab is database injection, where you can manipulate a database lookup query to log in as another user by controlling the input that's directly used in the query - but for now, let's focus on HTML injection (which is client-side).

When a user has control of how their input is displayed, they can submit HTML (or JavaScript) code, and the browser will use it on the page, allowing the user to control the page's appearance and functionality.



The image above shows how a form outputs text to the page. Whatever the user inputs into the "What's your name" field is passed to a JavaScript function and output to the page, which means if the user adds their own HTML or JavaScript in the field, it's used in the sayHi function and is added to the page - this means you can add your own HTML (such as a `<h1>` tag) and it will output your input as pure HTML.

The general rule is never to trust user input. To prevent malicious input, the website developer should sanitise everything the user enters before using it in the JavaScript function; in this case, the developer could remove any HTML tags.

Q1) View the website on this task and inject HTML so that a malicious link to <http://hacker.com> is shown.

Answer: HTML_INJECTION

HTML Injection Input: `<a href="http://hacker.com">Malicious Link</a>`